

Хабр Карьера

Курсы для роста в профессии

Получите навыки, за которые платят больше

Хабр



Чему научиться в этом году?



32K+

Охват за 30 дней

Ростелеком

Крупнейший провайдер цифровых услуг и решений

105,96 Рейтинг

53 931 Подписчики

[Подписаться](#)

ErikNas

15 часов назад

Эксперименты с распараллеливанием Java-автотестов

[🔗 Средний](#) [🕒 11 мин](#) [👁 6.6K](#)

Блог компании Ростелеком, Java*, Тестирование IT-систем*, Программирование*

[Тutorial](#)

Разрабы выкатили фикс и нужно прогнать регресс на стенде. Открываешь CI, запускаешь тесты и спустя каких-то пару часов можно понять катим мы релиз или нет. Деплой заблокирован, коллеги ждут мердж, а ты - тесты. А если нужно прогонять регресс каждый день или несколько раз в день?

Стандартный совет в таких случаях - "запусти тесты параллельно".
Окей, а как именно запускать?

В JUnit5 свой механизм распараллеливания. В Gradle - свой. А еще можно комбинировать. И конечно можно просто все сломать, если тесты начнут гонку за общие ресурсы.

Я провёл серию экспериментов: 5 видов сценариев, 4 режима запуска, 3 итерации для верности. Измерял не только скорость, но и корректность, чтобы не получилось "печатаю очень быстро, но получается какая-то ерунда".

В этой статье покажу:

- как настроить параллелизм в JUnit5 и Gradle (с примерами конфигов);
- что ломается, если включить параллелизм без подготовки;
- как изолировать БД и файловую систему для параллельных тестов;
- какой режим дает лучший баланс скорости и надежности.

Только JUnit5 + Gradle. Без Maven, без TestNG, без специфики CI-платформ.

TL;DR

Для самых нетерпеливых - сводная таблица результатов. Прогон 50 тестов по 6 секунд каждый.

Сценарий	Последовательно	JUnit (1x4)	Gradle (4x1)	Hybrid (2x2)
Чистые тесты	302 с	79 с (3.80x)	91 с (3.29x)	79 с (3.78x)
БД (без транзакций)	302 с	7 с (FAIL)	69 с (FAIL)	7 с (FAIL)
БД (с транзакциями)	302 с	79 с (3.80x)	92 с (3.29x)	80 с (3.78x)
Файлы (без синхр.)	302 с	79 с (flaky)	92 с (FAIL)	79 с (FAIL)
Файлы (с синхр.)	301 с	79 с (3.80x)	92 с (3.28x)	79 с (3.79x)

Методика: 5 сценариев x 4 режима

	Sequential	JUnit (1x4)	Gradle (4x1)	Hybrid (2x2)
base 50 чистых тестов	passed	passed	passed	passed
database_broken БД без транзакций	passed	FAILED	flaky	FAILED
database_transactional БД с транзакциями	passed	passed	passed	passed
filesystem_broken Файл без синхронизации	passed	flaky	FAILED	FAILED
filesystem_fixed Файл с ReentrantLock + FileLock	passed	passed	passed	passed

✓ — все тесты прошли ~ — flaky, периодически падает X — стабильно падает

Если коротко:

- Hybrid 2x2 (2 процесса x 2 потока) - универсальный выбор. Стабильно ~3.8x на всех сценариях.
- Включить параллелизм без изоляции ресурсов = гарантированные падения или, что хуже, скрытая потеря данных.
- Gradle "честнее" JUnit в некоторых тестах и чаще падает, там где JUnit может прятать проблему.

Если интересны подробности, то поехали.

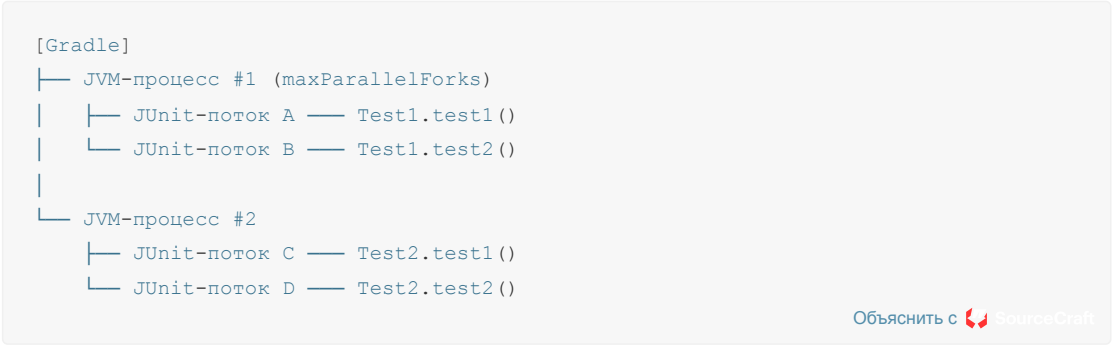
Потоки и процессы: два уровня параллелизма

Прежде чем крутить настройки попробуем вспомнить как это устроено в Java-мире. Параллельный запуск тестов может работать на двух уровнях, и они принципиально отличаются.

JUnit 5 Parallel Execution - потоки внутри одной JVM. Тесты живут внутри одного процесса, делят общую кучу (heap) и видят одни и те же статические переменные. Затраты минимальные. Поток создать дёшево, но и изоляция тоже на минимуме.

Gradle maxParallelForks - отдельные JVM-процессы. Gradle запускает n-копий Java-машины и каждая получает свою порцию тестовых классов. Полная изоляция памяти, но запуск каждой JVM будет потреблять ресурсы.

Гибридный режим - совмещение описанных выше вариантов. Запускается несколько процессов и внутри каждого процесса запускаются потоки. Выглядит это примерно вот так:



Каждый вариант имеет свои плюсы и минусы. Но в целом картина примерно следующая

	JUnit (потоки)	Gradle (процессы)
Изоляция памяти	Нет	Полная
Static-поля	Общие	Отдельные в каждой JVM
Файловая система	Общая	Общая
БД	Общая	Общая
Затраты	Минимальные	Ресурсы на запуск JVM
Лучше для	Чистых unit-тестов	Интеграционных тестов

Как это работает под капотом

Прежде чем переходить к настройкам попробуем посмотреть как это должно работать в теории на уровне JVM. Без этого можно собирать грабель.

Потоки в JVM - это потоки операционной системы

Каждый `Thread` в Java - это настоящий поток в операционной системе (классическая [модель 1:1](#), которая используется в HotSpot). Когда JUnit запускает 4 параллельных теста, то ОС создаёт 4 реальных потока и [планировщик ОС](#) распределяет их по ядрам CPU.

Мы действительно получаем параллелизм, а не просто конкурентность. Если конечно не запускаем потоков больше, чем у нас ядер у процессора. Когда потоков все же больше, чем ядер, то ОС переключается между ними. Каждое [переключение](#) это сохранение и восстановление состояния. Это и есть наши затраты.

Общая память

Потоки внутри одной JVM делят одну кучу (heap) и одно пространство классов (Metaspace). Из этого вытекает несколько неприятных следствий.

Static-поля общие для всех потоков. Если один тест пишет в `static int counter`, то другой поток видит изменения. Правда без гарантий. [Java Memory Model \(JMM\)](#) определяет правила [happens-before](#): без `volatile`, `synchronized` или `Lock` один поток может не увидеть изменения, сделанные другим. На x86 это стреляет реже так как архитектура имеет [сильную модель памяти](#), но на ARM - шансы возрастают.

И еще момент, который часто путают: Thread-safe != test-safe. `AtomicInteger` потокобезопасен, но если два теста пишут в один счетчик, то результат будет непредсказуем. Потокобезопасность класса не означает, что тесты с ним можно запускать параллельно.

Процессы

Когда Gradle запускает 4 форка, то каждый это отдельный java-процесс со своей кучей, GC, JIT-компилятором, static-полями и пространством имён потоков.

Общими остаются файловая система и сеть. Поэтому Gradle-форки не страдают от проблем с static-полями, но конкуренция за доступ к файлам или БД никуда не девается.

Закон Амдала

Теоретический максимум ускорения определяется [законом Амдала](#):

$$\text{Ускорение} = 1 / (a + (1 - a) / p)$$

[Объяснить с](#) 

Где **a** - доля последовательного кода, **p** - количество параллельных потоков.

В моих тестах `Thread.sleep` - полностью параллелизуемая операция ($a = 0$), следовательно теоретический максимум 4x на 4 потоках. Реально же получается 3.8x. Оставшаяся часть как раз последовательные операции: инициализация JUnit, синхронизация Gradle, работа GC.

В реальных проектах последовательная часть сильно больше: spring-контекст, миграции БД, прогрев кэшей и т.д. Поэтому на реальных данных даже ускорение в 2-3x на большом тестовом наборе это однозначно победа.

Немного про Thread.sleep

Может показаться, что я выбрал не самый удачный пример в своих тестах. Но на деле большинство тестов в типичном проекте ждут ответа от БД, от HTTP-сервера, от очереди сообщений. `Thread.sleep` имитирует это ожидание: поток заблокирован, ядро CPU свободно и другой поток может занять его без конкуренции.

Параллелизм в JUnit 5

Настройка JUnit 5 на параллельное выполнение это буквально несколько строк.

Файл `src/test/resources/junit-platform.properties` :

```
junit.jupiter.execution.parallel.enabled=true
junit.jupiter.execution.parallel.mode.default=concurrent
junit.jupiter.execution.parallel.config.strategy=fixed
junit.jupiter.execution.parallel.config.fixed.parallelism=4
```

[Объяснить с 🍷](#)

Четыре параметра и тесты уже можно запускать в 4 потока. `mode.default=concurrent` означает, что и классы и методы внутри классов будут выполняться параллельно. Если хочется параллелить только классы, то можно написать вот так:

```
junit.jupiter.execution.parallel.mode.default=same_thread
junit.jupiter.execution.parallel.mode.classes.default=concurrent
```

[Объяснить с 🍷](#)

Для отдельных тестов или классов можно управлять через аннотацию

`@Execution(CONCURRENT)` или `@Execution(SAME_THREAD)` .

В своем эксперименте настройки JUnit передаю через `systemProperties` в `build.gradle` . Так было банально удобнее для запуска разных Gradle-задач с разными настройками, чем хардкодить properties-файлы и подкидывать их.

Параллелизм в Gradle

Gradle предлагает свой механизм `maxParallelForks` . Он запускает тестовые классы в отдельных JVM-процессах.

```
tasks.register('testGradleParallel', Test) {
    useJUnitPlatform()
    maxParallelForks = 4 // 4 отдельных JVM-процесса
    systemProperties = [
        'junit.jupiter.execution.parallel.enabled': 'false' // JUnit НЕ параллелит
    ]
}
```

[Объяснить с 🍷](#)

Главное не путать `maxParallelForks` с флагом `--parallel`. Последний параллелит сборку, а не тесты. Для проекта, который состоит из одного модуля флаг `--parallel` ничего не даст.

Комбинированный режим

Тут просто собираем обе настройки вместе

```
tasks.register('testHybrid2x2', Test) {
    useJUnitPlatform()
    maxParallelForks = 2 // 2 процесса
    systemProperties = [
        'junit.jupiter.execution.parallel.enabled' : 'true',
        'junit.jupiter.execution.parallel.mode.default' : 'concurrent',
        'junit.jupiter.execution.parallel.mode.classes.default' : 'concurrent',
        'junit.jupiter.execution.parallel.config.strategy' : 'fixed',
        'junit.jupiter.execution.parallel.config.fixed.parallelism' : '2' // 2 потока
    ]
    // Итого: 2 процесса x 2 потока = 4 параллельных теста
}
```

[Объяснить с](#) 

Методика испытаний

Что измеряем

- 50 тестов (10 классов x 5 методов), каждый выполняет `Thread.sleep(6000)` — имитация 6 секунд работы.
- Базовое время: $50 \times 6 = 300$ секунд последовательно.
- 3 прогона каждого режима, берём среднее. Смотрим разброс между прогонами.

На чем гоняем

Основная моя машинка это HP ProBook 430 G8.

```
OS:      ALT Workstation K 11.2 (Nemorosa) x86_64
Kernel:  Linux 6.12.74-6.12-alt1
CPU:     11th Gen Intel(R) Core(TM) i7-1165G7 (8) @ 4.70 GHz
Memory:  15.31 GiB
Java:    21.0.10 (OpenJDK)
Gradle:  8.10
```

[Объяснить с](#) 

5 сценариев

1. base - чистые тесты без общих ресурсов
2. database_broken - тесты с общим БД (Commit + DELETE)

3. database_fixed - тесты с общим БД, но уже есть изоляция транзакций (ThreadLocal + rollback)
4. filesystem_broken - простая запись в общий файл
5. filesystem_fixed - синхронизированная запись (ReentrantLock + FileLock)

4 режима

1. Sequential - 1 процесс, 1 поток
2. JUnit Parallel - 1 процесс, 4 потока
3. Gradle Parallel - 4 процесса, 1 поток
4. Hybrid 2x2 - 2 процесса, 2 потока в каждом

Базовые тесты

Начнем с идеального случая. У тестов нет общих ресурсов. Никаких БД, файлов, устройств, кэшей и т.п.

```
public abstract class CleanTestTemplate {  
    protected static final long TEST_DURATION_MS = 6000;  
  
    @Test  
    void test01() throws InterruptedException {  
        Thread.sleep(TEST_DURATION_MS);  
    }  
    // ... test02-05  
}
```

Объяснить с 

Десять классов наследуют этот шаблон. 50 тестов. 300 секунд последовательно.

Результаты

Режим	Время	Ускорение
Sequential	302 с	1.00x
JUnit (1x4)	79 с	3.80x
Gradle (4x1)	91 с	3.29x
Hybrid (2x2)	79 с	3.78x

Ускорение 3.8x из теоретически возможных 4x. Эффективность 95%. Gradle стабильно медленнее JUnit и Гибридного режима на ~12 секунд. Причиной являются затраты на запуск 4 отдельных JVM-процессов.

Вывод здесь можно сделать такой, что если тесты не делят ресурсы, то просто включай JUnit Parallel.

Тесты с БД

Переходим к более интересным вещам.

Что может пойти не так?

Допустим у нас есть проект с тестами написанными для последовательного запуска. Каждый тест:

1. Вставляет запись в БД (INSERT)
2. Проверяет, что в таблице есть его данные (SELECT count(*))
3. Делает полезную работу (в моих тестах на этом месте просто Thread.sleep)
4. Удаляет за собой данные (DELETE в [@AfterEach](#))

Последовательно все будет работать идеально. Потом включаем параллелизм и привет:

```
@ExtendWith(NaiveDbSetup.class)
public abstract class NaiveTestTemplate {

    private void testUserOperation(int methodId) throws SQLException, InterruptedException {
        int userId = getClassOffset() + methodId;
        Connection conn = NaiveDbSetup.getConnection();

        // Вставляем пользователя (autoCommit=true -> сразу виден ВСЕМ)
        insertUser(conn, userId, userName, balance);

        // Проверяем: "в таблице должна быть только моя запись"
        int totalUsers = NaiveDbSetup.countAllUsers(conn);
        assertThat(totalUsers)
```



```

        .as("Ожидаем <= 1 запись, но другие тесты уже вставили свои")
        .isLessThanOrEqualTo(1); // <-- Ошибка при параллелизме

        Thread.sleep(TEST_DURATION_MS);
        // afterEach: DELETE... но уже поздно
    }
}

```

[Объяснить с](#) 

Основная проблема в `autoCommit=true`. Это поведение JDBC по умолчанию. Каждый INSERT мгновенно виден всем потокам и процессам. Тест А вставил запись, тест Б делает `SELECT count(*)` и видит не одну запись, а две. Assert падает.

Результаты broken-сценария

Режим	Время	Результат
Sequential	302 с	50/50 passed
JUnit (1x4)	7 с	49/50 FAILED
Gradle (4x1)	69 с	35-41/50 FAILED
Hybrid (2x2)	7 с	49/50 FAILED

В Gradle количество упавших тестов нестабильно. Чем меньше упало, тем дольше прогон. Это связано с тем, что Gradle-форки имеют полную изоляцию процессов, что снижает вероятность коллизии.

Транзакционная изоляция

Да, БД у нас одна, но мы можем заставить каждый поток работать в своей транзакции. Транзакция будет откатываться после теста. Никто не видит чужих данных, так как они не закоммичены.

```
public class TransactionalDbSetup
    implements BeforeAllCallback, BeforeEachCallback, AfterEachCallback {

    private static final ThreadLocal<Connection> threadLocalConnection = new ThreadLocal<>();

    @Override
    public void beforeEach(ExtensionContext context) throws Exception {
        Connection conn = DriverManager.getConnection(DB_URL);
        conn.setAutoCommit(false); // Начало транзакции
        threadLocalConnection.set(conn);
    }

    @Override
    public void afterEach(ExtensionContext context) throws Exception {
        Connection conn = threadLocalConnection.get();
        if (conn != null && !conn.isClosed()) {
            conn.rollback(); // Откат всех изменений
            conn.close();
            threadLocalConnection.remove();
        }
    }
}
```

[Объяснить с](#) 

Тут работает связка из `ThreadLocal<Connection>` и каждый поток получает свое соединение, `setAutoCommit(false)` и данные не видны другим соединениям до коммита, а коммита и не будет, `rollback()` после теста все изменения откатываются и БД снова чистая.

Результаты transactional-сценария

Режим	Время	Ускорение
Sequential	302 с	1.00x
JUnit (1x4)	79 с	3.80x
Gradle (4x1)	92 с	3.29x
Hybrid (2x2)	80 с	3.78x

Все 50 тестов, все 3 прогона, все 4 режима - прошли. Ускорение практически такое же, как на чистых тестах.

В реальных проектах вместо ручного `ThreadLocal` + `rollback` чаще используют `@Transactional` от Spring Boot (под капотом тот же `setAutoCommit(false)` + `rollback()`, но декларативно) или Testcontainers с отдельной БД на каждый процесс.

Тесты с файлом

С файловой системой всё хуже, чем с БД. Тут нет транзакций, нет `rollback`, нет уровней изоляции - только файл и несколько потоков, которые хотят в него что-то записать.

Пишем в файл как обычно


```
        channel.write(ByteBuffer.wrap(bytes));
    }
}
writesAttempted++;
} finally {
    jvmLock.unlock();
}
}
}
```

Объяснить с 

Результаты fixed-сценария

Режим	Время	Ускорение	Целостность данных
Sequential	301 с	1.00x	100%
JUnit (1x4)	79 с	3.80x	100%
Gradle (4x1)	92 с	3.28x	100%
Hybrid (2x2)	79 с	3.79x	100%

Тут все чётенько.

Итоги испытаний

Сводная таблица (финальный прогон, 60 запусков)

Сценарий	Sequential	JUnit (1x4)	Gradle (4x1)	Hybrid (2x2)
base	302 с	79 с / 3.80x	91 с / 3.29x	79 с / 3.78x
database_broken	302 с	FAIL (49/50)	FAIL (35-41/50)	FAIL (49/50)
database_transactional	302 с	79 с / 3.80x	92 с / 3.29x	80 с / 3.78x
filesystem_broken	302 с	79 с / flaky	92 с / FAIL	79 с / FAIL
filesystem_fixed	301 с	79 с / 3.80x	92 с / 3.28x	79 с / 3.79x

Данные по стабильности

Режим	Прогон #1	Прогон #2	Прогон #3	Разброс
JUnit	3.58-3.75x	3.70-3.80x	3.80x	<=0.05x
Gradle	3.25-3.26x	3.27-3.29x	3.28-3.29x	<=0.04x
Hybrid	3.73-3.75x	3.78-3.79x	3.78-3.80x	<=0.05x

Результаты в целом сходятся. Разброс между отдельными прогонами внутри одного запуска не значительный.

Какой вариант лучше выбрать?

Сценарий	Рекомендация	Почему
Чистые unit-тесты	JUnit 5 (1x4)	Минимум конфигурации
Интеграционные тесты	Gradle (4x1)	Изоляция через процессы
Смешанный проект	Hybrid 2x2	Баланс изоляции и затрат
Много классов, мало методов	Gradle (4x1)	Параллелизм на уровне классов

Мало классов, много методов	JUnit (1x4)	Параллелизм на уровне методов
Есть общие ресурсы	Сначала изолировать	Без изоляции — только хуже

Выводы

Гибридный вариант параллелизма

Два процесса по два потока показали стабильный результат по испытаниям на всех сценариях. Быстрый как JUnit, но не маскирует проблемные тесты. Наверное самый универсальный выбор. Если что меня поправят в комментариях.

Включить `parallel.enabled=true` это наверное секунд 10, а вот подготовить тесты к параллельному запуску может отнять тонну времени. Конечно все зависит от проекта, но лучше перепроверить тесты перед переходом.

Спасибо, что дочитали до конца. Если интересно, то могу рассказать как мы насобирали грабель при распараллеливании playwright тестов. Если нужны полные исходники, то тоже маякните - залью в репку.

Теги: [gradle](#), [автотесты](#), [junit5](#), [параллельные тесты](#), [ускорение тестирования](#)

Хабы: [Блог компании Ростелеком](#), [Java](#), [Тестирование IT-систем](#), [Программирование](#)



Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



32K+

Охват за 30 дней

Ростелеком

Крупнейший провайдер цифровых услуг и решений

[Хабр Карьера](#) [Telegram](#)



4K+

14

Охват за 30 дней

Карма

Ирек Насибуллин [@ErikNas](#)

qa

Подписаться



Поток Тестирование доступен 24/7 благодаря поддержке друзей Хабра

Хабр Курсы для всех

РЕКЛАМА

Практикум, Хекслет, SkyPro, авторские курсы — собрали всех и попросили скидки. Осталось выбрать!

Перейти

[Перейти в поток Тестирование](#)

 [Комментарии 2](#)

Публикации

[ЛУЧШИЕ ЗА СУТКИ](#)

[ПОХОЖИЕ](#)



the_bat

23 часа назад

Ремонт блока питания с Power Delivery. 470 граммов электроники

 **Простой**

 4 мин

 16K

 +96

 30

 27



interpres

18 часов назад

Электроинструмент становится хуже, и это делается намеренно

 **Простой**

 11 мин

 27K

Обзор

Перевод

 +71

 45

 83



prohronus

19 часов назад

Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере

 5 мин

 9.3K

Кейс

 +51

 9

 9



alizar

22 часа назад

Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц

 **Простой**

 7 мин

 12K

Обзор

 +37

 59

 18



AlexSerbul

23 часа назад

Программирование с AI-ассистентом — похороните меня под плитусом

🗨 Средний

🕒 14 мин

👁 9.9K

Мнение

👍 +28

🔖 33

💬 11



TrexSelectel

19 часов назад

Linux 7.0 и что изменилось в ядре после очередного цикла разработки

🕒 5 мин

👁 8.8K

Обзор

👍 +27

🔖 10

💬 0



mariklolik

18 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

🗨 Средний

🕒 7 мин

👁 7.5K

Кейс

👍 +23

🔖 23

💬 6



nick_dyuba

22 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и TіріTіr

🕒 5 мин

👁 7.1K

Recovery Mode

👍 +22

🔖 8

💬 5



Mimizavr

23 часа назад

«Великое очищение» в работе с контентом: что осталось от роли редактора

🕒 11 мин

👁 7K

👍 +17

🔖 9

💬 7



infgotoinf

10 часов назад

Perl — зря забытый язык программирования?

🕒 11 мин

👁 8.5K

Из песочницы

👍 +16

🔖 13

💬 9

Что нам стоит IaaS построить: как мы собирали прочный фундамент для облака

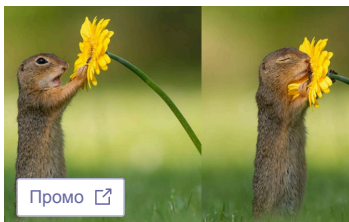
Турбо

Показать еще

МИНУТОЧКУ ВНИМАНИЯ



Удалёнка в фарме: большие данные и никакой магии



Запахло весной и скидками в Промокодусе



Собери облачный пазл и выиграй призы

ИНФОРМАЦИЯ

Сайт

www.company.rt.ru

Дата регистрации

1 февраля 2012

Дата основания

23 сентября 1993

Численность

свыше 10 000 человек

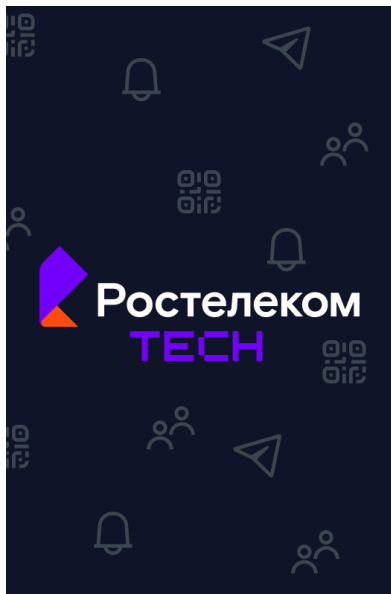
Местоположение

Россия

Представитель

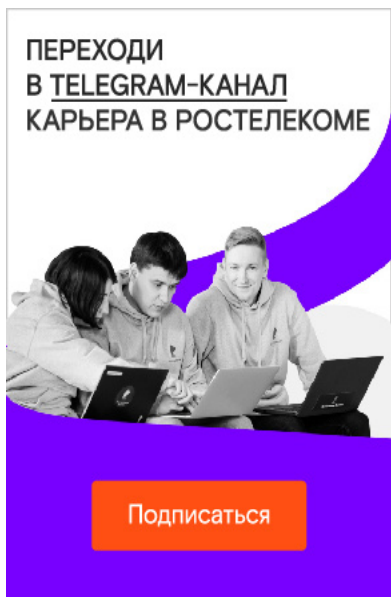
[Vatuhaa](#)

ВИДЖЕТ



Рассказываем про ИТ-сторону «Ростелекома» и внутреннюю жизнь его лампового айтишного комьюнити.

ВИДЖЕТ



Разрушаем мифы о работе в крупных компаниях, и показываем, какие проекты запускает команда одного из крупнейших ИТ-кластеров в стране

ССЫЛКИ

[Витрина цифровых продуктов](#)

БЛОГ НА ХАБРЕ

15 часов назад

Эксперименты с распараллеливанием Java-автотестов

6.6K 2

14 апр в 17:54

Мой путь к качественному сну после перехода на удаленку: изучаю, тестирую, применяю

6.3K 8

9 апр в 14:56

Аврора Центр: как мы помогаем банкам собирать биометрию на отечественных устройствах

6.5K 8

31 мар в 14:23

UI + API как единый интеграционный контур

5.3K 0

26 мар в 20:55

В инфобез со студенческой скамьи: как пройти фейсконтроль рекрутера, если ты джун

7.4K 5

ИСТОРИИ



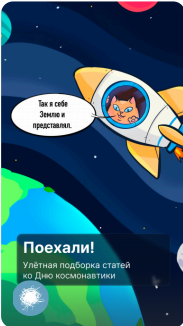
Годнота из блогов компаний



Только 10% решат эту головоломку



Хочу 450K



Вперёд к звёздам



Там на неведомых дорожках...



Самое время поспать

Ваш аккаунт

Войти
Регистрация

Разделы

Статьи
Новости
Хабы
Компании
Авторы
Песочница

Информация

Устройство сайта
Для авторов
Для компаний
Документы
Соглашение
Конфиденциальность

Услуги

Корпоративный блог
Медийная реклама
Нативные проекты
Образовательные программы
Стартапам



Настройка языка

Техническая поддержка

© 2006–2026, Habr